MACHINE LEARNING TECHNICAL ASSESSMENT

House Prices - Advanced Regression Techniques

Technical Report

This report documents an end-to-end regression workflow for predicting residential house sale prices using the Kaggle House Prices - Advanced Regression Techniques dataset. It summarizes the exploratory analysis, feature engineering, preprocessing, model comparison, validation error review, and final Kaggle submission.

TRAINING ROWS	TEST ROWS	BEST VALIDATION LOG-RMSE	KAGGLE SCORE
1,460	1,459	0.134697	0.12800

Executive Summary

The original dataset contained 1,460 training observations with 81 columns (including the target) and 1,459 test observations with 80 columns. The target, `SalePrice`, was modelled on a `log1p` scale to align model validation with the competition's logarithmic RMSE objective. A reproducible 80/20 split (`random_state=42`) was used for validation.

Four approaches were compared. Ridge Regression (alpha=10) obtained the lowest validation Log-RMSE of 0.134697, ahead of Gradient Boosting (0.140550), Random Forest (0.145227), and a mean Dummy Baseline (0.433244). Ridge Regression was then fitted on the full labelled training dataset to generate the submission file. The submitted Kaggle result reported a score of 0.12800.

Project workflow

01	02	03	04	05
EDA	Preprocessing	Experiments	Final model	Kaggle submit

Competition: Kaggle - House Prices: Advanced Regression Techniques

1. Problem Understanding and Exploratory Data Analysis

Problem definition

The task is a supervised regression problem: learn from property characteristics in the training dataset and predict the sale price for each row in the test dataset. The target column is `SalePrice`. The competition submission requires the columns `Id` and `SalePrice`.

Dataset overview

Item	Observed value	Interpretation
Training data	1,460 rows x 81 columns	Includes SalePrice target.
Test data	1,459 rows x 80 columns	SalePrice is not supplied for prediction rows.
Numerical columns	39 in EDA listing	Numeric columns were used for descriptive statistics and correlations; the count included the added LogSalePrice column.
Categorical columns	43 object columns	Categorical features required missing-value handling and encoding.

Target analysis

The training target was right-skewed. The observed SalePrice summary was: mean 180,921.20; median 163,000; standard deviation 79,442.50; minimum 34,900; and maximum 755,000. The notebook also visualized `log1p(SalePrice)`, which produced a more symmetric distribution and was used as the modelling target.

Missing data

Feature	Missing values	Feature	Missing values
PoolQC	1,453	**MiscFeature**	1,406
Alley	1,369	**Fence**	1,179
MasVnrType	872	**FireplaceQu**	690
LotFrontage	259	**Garage-related fields**	81 each for reported fields

Missingness varied considerably across fields. For garage, basement, alley, pool, and other property attributes, missing values may indicate absence of that feature, while other fields may represent unknown values. The preprocessing pipeline uses a consistent numerical and categorical imputation rule; a domain-aware missingness strategy could be explored in future iterations.

Relationships and outliers

The strongest numerical relationships with SalePrice included OverallQual (correlation 0.791), GrLivArea (0.709), GarageCars (0.640), GarageArea (0.623), TotalBsmtSF (0.614), and 1stFlrSF (0.606). Scatter plots were created for GrLivArea vs SalePrice and OverallQual vs SalePrice, and a box plot compared SalePrice across OverallQual levels. These checks help identify associations and unusual observations; correlation does not establish causation.

2. Preprocessing and Feature Engineering

Validation design

The labelled data was split into an 80% training subset and a 20% validation subset using `train_test_split` with `random_state=42`. This resulted in 1,168 training rows and 292 validation rows. The target used for training and validation was `log1p(SalePrice)`. The split makes model scores comparable on the same held-out rows, but it is a single split rather than cross-validation.

Missing values and encoding

Feature type	Processing	Reason
Numerical	Median imputation, then StandardScaler	Provides a simple missing-value strategy and puts numeric scales on a comparable scale for Ridge.
Categorical	Constant imputation with value "Missing", then `OneHotEncoder(handle_unknown="ignore")`	Encodes category labels and tolerates categories not seen in the training partition.
Combined workflow	ColumnTransformer inside a scikit-learn Pipeline	Fits preprocessing only on the training partition during model validation, reducing leakage risk.

Engineered features

Feature	Definition / purpose
`TotalSF`	TotalBsmtSF + 1stFlrSF + 2ndFlrSF; approximates total finished and basement area.
`TotalBathrooms`	Full baths plus 0.5 times half baths, including basement bathrooms.
`TotalPorchSF`	Sum of OpenPorchSF, 3SsnPorch, EnclosedPorch, ScreenPorch, and WoodDeckSF.
`HouseAge`	YrSold - YearBuilt.
`RemodAge`	YrSold - YearRemodAdd.
`OverallQual_SF`	OverallQual multiplied by GrLivArea, combining a quality rating with living area.

The same feature engineering function was applied to training and test rows. After adding six engineered features, the training data contained 87 columns and the test data contained 86 columns. The SalePrice target remains only in the training dataset.

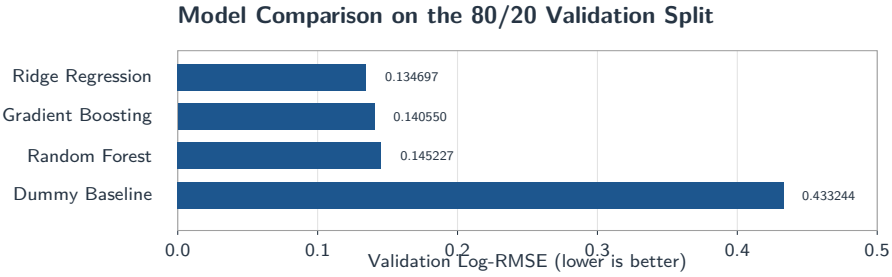
Baseline

Ridge Regression with alpha=10 was used as the initial practical baseline after preprocessing. Its validation Log-RMSE was 0.134697. The result provides a reference point against which the Dummy baseline and tree-based models were compared.

3. Model Experiments and Validation Results

Model comparison

Four approaches were trained and scored on the same validation split. The evaluation computed RMSE between the log-transformed target and model predictions; lower values are better.



Model	Validation Log-RMSE	Decision
Ridge Regression (alpha=10)	0.134697	Selected final model; lowest validation error.
Gradient Boosting	0.140550	Second-best result among tested models.
Random Forest	0.145227	Outperformed the dummy baseline, but not Ridge on this split.
Dummy Baseline (mean)	0.433244	Simple reference model; substantially higher error.

Interpretation

Ridge Regression outperformed both tested tree-based models on this particular holdout split. The Dummy baseline result indicates that the learned models captured useful signal beyond a constant prediction. Because only one random split was used, this ranking should be treated as evidence for model selection in this experiment, not proof that Ridge will always outperform the alternatives. Cross-validation and hyperparameter tuning are reasonable next steps.

Validation error review

The Ridge validation predictions were reviewed by ranking observations on absolute log-scale error. The largest error examples included the following cases. Predicted prices are converted back from the log scale for readability; the error value itself remains on the log scale.

Competition Id	Actual price	Predicted price	Abs. log error
31	$40,000	$84,887	0.7524
813	$55,993	$99,814	0.5781
917	$35,311	$60,820	0.5437
1433	$64,500	$105,806	0.4949
899	$611,657	$440,103	0.3292

Several lower-priced homes were overpredicted, while a high-priced home (Id 899) was underpredicted. This may reflect atypical properties or features not fully captured by the model, but these examples alone do not establish the cause. Further residual plots and subgroup diagnostics would be useful.

4. Final Model, Kaggle Submission and Conclusion

Final model

Ridge Regression (alpha=10) was selected based on the lowest validation Log-RMSE among the four tested approaches. The selected pipeline was fitted on all 1,460 labelled training rows after feature engineering and preprocessing. Predictions were then generated for the 1,459 rows in the competition test dataset. The log-scale predictions were converted back to the price scale with `expm1`, and values were constrained to be non-negative.

Submission file checks

Check	Observed result
Rows	1,459 test predictions
Columns	Id, SalePrice
Missing predictions	None detected in the notebook checks
Negative predicted prices	None after applying a non-negative lower bound
Kaggle submission	Submitted successfully; score shown as 0.12800

Result

KAGGLE SCORE REPORTED BY THE SUBMISSION PAGE

0.12800

The score above is the result reported in the user-provided Kaggle submission screenshot. The report does not infer a leaderboard rank, percentile, or broader performance claim from the score alone.

Reproducibility and repository contents

The repository is organized to include four notebooks in order (EDA, preprocessing, model experiments, and final model), a requirements file, reusable source modules for preprocessing, feature engineering, models, and evaluation, the model-results CSV, the final submission CSV, EDA figure files, and this technical report. The notebooks use the Kaggle competition data as input. To reproduce the work, attach the same competition input, execute the notebooks in order, and confirm that the recorded model scores and submission checks match.

Known limitations and future work

- Validation used one 80/20 random split; K-fold cross-validation would give a more stable estimate of generalization.
- The preprocessing strategy is deliberately simple. Domain-aware missing-value treatment and feature selection could improve it.
- A more systematic residual review should inspect large errors by feature group and test whether any individual cases dominate the model error.
- Further hyperparameter tuning and additional gradient-boosted models could be compared on the same validation protocol before selecting a final estimator.

Conclusion

This project completed an end-to-end house-price regression workflow: data exploration, missing-value handling, categorical encoding, six engineered features, four-model comparison, validation error review, final model fitting, and a successful Kaggle submission. Ridge Regression was the strongest model in the recorded validation comparison, and the submission page reported a score of 0.12800. The work is reproducible through the project notebooks and supporting files, with cross-validation and deeper residual diagnostics identified as the most useful improvements.